Cloud Computing Project Proposal

AI Powered DevOps Assistant

Team

Swarna Sweth, Sunidhi Suri, Furkan Gokmenoglu, Rohan Bhalla

Goal

The goal of this project is to build a natural-language DevOps AI assistant that allows the user to interact using a chatbot style and have their requests turned into validated, deployable infrastructure-as-code and CI/CD configurations on AWS. A continuously updated retrieval pipeline keeps generated configurations aligned with the latest tool versions and documentation, which reduces the expertise and time needed for common DevOps setup tasks.

Features

The features of this project are two fold. Firstly, providing users with the ability to ask questions about documentation and libraries by having the assistant be up to date with new tools, versions, and docs, and use that knowledge to answer and explain. This will be a RAG based AI chatbot with the following features:

- Scheduled ingestion of docs and release notes
- Version-aware generation (current Node LTS, latest action versions)
- Q&A that cites actual documentation
- Explaining scan results and build failures, with fixes pointing to real, current versions

The second set of features offered use the chat interface for plain English DevOps requests along with follow up questions that the assistant will ask to clarify for missing parameters and specifications. Similar to what we've seen in the demos, there will be intent classification of the user's chat message and accordingly based on user confirmation the request will be fulfilled to generate their requested artifacts.

Support for the following tasks will be provided:

- CI/CD setup command for a Node.js app (GitHub Actions support)
- Containerizing an app by generating dockerfile and deploying it
- Deploying Lambda functions
- Creating a Kubernetes (EKS) cluster with autoscaling
- Deploying a static site on AWS
- A plan → review → approved workflow that shows generated files and a summary
- Sandboxed execution with live deployment logs and a one-click teardown.
- An audit history of every request, the generated artifacts, and the deploy outcome.

AWS Resources

Feature	AWS Service
LLM and Embedding generation for RAG	AWS Bedrock
Vector DB for RAG	Bedrock Knowledge Bases + S3 Vectors
Raw/original documentation storage	S3 storage
Scheduled ingestion to keep the knowledge base updated (A scheduled job)	EventBridge Scheduler + Lambda
API backend	API Gateway HTTP API + Lambda
Workflow orchestration for the steps that a request has to go through	Step Functions
Kubernetes deployment	EKS + ECR
Monitoring	Cloudwatch
Frontend Hosting	S3 + CloudFront
Access control	IAM
Secrets/credentials	Secrets Manager

Stretch Goals

Apart from the features outlined above, some additional functionalities that could be implemented if time permits are:

- **Slack or other chat integration:** Make an integration in which users can directly delegate tasks to the InfraPilot by doing “@infrapilot” in slack
- **Vulnerability checking:** Use Amazon Inspector to scan container images stored in ECR and return vulnerability information to the assistant. An automatic validation and self-correction loop with retries before anything reaches the user.
- **Cost estimation**: Before deployment, InfraPilot can use the AWS Price List API to estimate the expected cost of the generated infrastructure.
- **Monitoring dashboard:** Automatically create a CloudWatch dashboard for deployed applications to display application logs, CPU/memory utilization, deployment status, and other such metrics